
AP and SS Axioms

- **Definition: Action Precondition Axiom**

An action precondition axiom is a sentence of the form:

$$(\forall x_1, \dots, x_n, s). Poss(A(x_1, \dots, x_n), s) \equiv \Pi_A(x_1, \dots, x_n, s),$$

where A is an n -ary action function, and Π_A is a formula that is uniform in s and whose free variables are among $x_1, \dots, x_n, s$.

- **Definition: Successor State Axiom** A successor state axiom for an $(n + 1)$ -ary fluent F is a sentence of the form:

$$(\forall a, s)(\forall x_1, \dots, x_n). F(x_1, \dots, x_n, do(a, s)) \equiv \Phi_F, \quad (5)$$

where Φ_F is a formula uniform in s , all of whose free variables are among $a, s, x_1, \dots, x_n$.

- Formulas uniform in s = Markov property. The future is determined by the present.
- We do not assume that successor state axioms have the exact syntactic form as those obtained earlier by combining Schubert and Pednault's ideas. The discussion there was meant to motivate one way that successor state axioms of the form (5) might arise, but nothing in the development that follows depends on that earlier approach.

Basic Action Theories

$$\mathcal{D} = \Sigma \cup \mathcal{D}_{ss} \cup \mathcal{D}_{ap} \cup \mathcal{D}_{una} \cup \mathcal{D}_{S_0}$$

where

- Σ are the foundational axioms for situations.
- $\mathcal{D}_{ss}$ is a set of successor state axioms.
- $\mathcal{D}_{ap}$ is a set of action precondition axioms.
- $\mathcal{D}_{una}$ is the set of unique names axioms for actions.
- $\mathcal{D}_{S_0}$ is a set of first order sentences with the property that S_0 is the only term of sort *situation* mentioned by the fluents of a sentence of $\mathcal{D}_{S_0}$. Thus, no fluent of a formula of $\mathcal{D}_{S_0}$ mentions a variable of sort *situation* or the function symbol *do*. $\mathcal{D}_{S_0}$ will play the role of the initial situation of the world (i.e. the one we start off with, before any actions have been “executed”).

Theorem 1 (Relative Satisfiability)

$\mathcal{D}$ is satisfiable iff $\mathcal{D}_{una} \cup \mathcal{D}_{S_0}$ is.

Regression

- **The Regressable Formulas.**

The essence of a regressable formula is that each of its *situation* terms is rooted at S_0 , and therefore, one can tell, by inspection of such a term, exactly how many actions it involves. It is not necessary to be able to tell what those actions are, just how many they are. In addition, when a regressable formula mentions a *Poss* atom, we can tell, by inspection of that atom, exactly what is the action function symbol occurring in its first argument position, for example, that it is a *move* action.

- Assume a background axiomatization that includes a set of successor state and action precondition axioms.

- **The Regression Operator: Simple Version**

W is a regressable formula of $\mathcal{L}_{sitcalc}$ that mentions no functional fluents.

1. Suppose W is an atom. Since W is regressable, there are four possibilities:

- (a) W is a situation independent atom. Then

$$\mathcal{R}[W] = W.$$

- (b) W is a relational fluent atom of the form $F(\vec{t}, S_0)$. Then

$$\mathcal{R}[W] = W.$$

- (c) W is a regressable *Poss* atom, so it has the form $Poss(A(\vec{t}), \sigma)$ for terms $A(\vec{t})$ and σ of sort *action* and *situation* respectively. Here, A is an action function

symbol of $\mathcal{L}_{sitcalc}$. Then there must be an action precondition axiom for A of the form

$$Poss(A(\vec{x}), s) \equiv \Pi_A(\vec{x}, s).$$

Assume that all quantifiers (if any) of $\Pi_A(\vec{x}, s)$ have had their quantified variables renamed to be distinct from the free variables (if any) of $Poss(A(\vec{t}), \sigma)$. Then

$$\mathcal{R}[W] = \mathcal{R}[\Pi_A(\vec{t}, \sigma)].$$

In other words, replace the atom $Poss(A(\vec{t}), \sigma)$ by a suitable instance of the right hand side of the equivalence in A 's action precondition axiom, and regress that expression. The above renaming of quantified variables of $\Pi_A(\vec{x}, s)$ prevents any of these quantifiers from capturing variables in the instance $Poss(A(\vec{t}), \sigma)$.

- (d) W is a relational fluent atom of the form $F(\vec{t}, do(\alpha, \sigma))$. Let F 's successor state axiom in $\mathcal{D}_{ss}$ be

$$F(\vec{x}, do(a, s)) \equiv \Phi_F(\vec{x}, a, s).$$

Assume that all quantifiers (if any) of $\Phi_F(\vec{x}, a, s)$ have had their quantified variables renamed to be distinct from the free variables (if any) of $F(\vec{t}, do(\alpha, \sigma))$. Then

$$\mathcal{R}[W] = \mathcal{R}[\Phi_F(\vec{t}, \alpha, \sigma)].$$

In other words, replace the atom $F(\vec{t}, do(\alpha, \sigma))$ by a suitable instance of the right hand side of the equivalence in F 's successor state axiom, and regress this formula. The above renaming of quantified variables of $\Phi_F(\vec{x}, a, s)$ prevents any of these quantifiers from cap-

turing variables in the instance $F(\vec{t}, do(\alpha, \sigma))$.

2. For non-atomic formulas, regression is defined inductively.

$$\mathcal{R}[\neg W] = \neg \mathcal{R}[W],$$

$$\mathcal{R}[W_1 \wedge W_2] = \mathcal{R}[W_1] \wedge \mathcal{R}[W_2],$$

$$\mathcal{R}[(\exists v)W] = (\exists v)\mathcal{R}[W].$$

- Intuitively, the regression operator eliminates *Poss* atoms in favour of their definitions as given by action precondition axioms, and replaces fluent atoms about $do(\alpha, \sigma)$ by logically equivalent expressions about σ as given by successor state axioms. Moreover, it repeatedly does this until it cannot make such replacements any further.
- Each $\mathcal{R}$ -step reduces the depth of nesting of the function symbol do in the fluents of W by substituting suitable instances of Φ_F for each occurrence of a fluent atom of W of the form $F(t_1, \dots, t_n, do(\alpha, \sigma))$. Since no fluent atom of Φ_F mentions the function symbol do , the effect of this substitution is to replace each such F by a formula whose fluents mention only the situation term σ , and this reduces the depth of nesting by one.

Theorem 2 (The Regression Theorem) *Suppose W is a regressable sentence of $\mathcal{L}_{sitcalc}$ that mentions no functional fluents, and $\mathcal{D}$ is a basic theory of actions. Then,*

$$\mathcal{D} \models W \quad \text{iff} \quad \mathcal{D}_{S_0} \cup \mathcal{D}_{una} \models \mathcal{R}[W].$$

Executable Action Sequences

- Consider a sequence $\alpha_1, \dots, \alpha_n$ of ground action terms.
- Problem: Determine whether this is executable. Is it the case that:

$$\mathcal{D} \models executable(do([\alpha_1, \dots, \alpha_n], S_0))$$

- Exercise:

$$\Sigma \models (\forall a_1, \dots, a_n).executable(do([a_1, \dots, a_n], S_0)) \equiv \bigwedge_{i=1}^n Poss(\alpha_i, do([\alpha_1, \dots, \alpha_{i-1}], S_0)).$$

Theorem 3 *Suppose that $\alpha_1, \dots, \alpha_n$ is a sequence of ground action terms of $\mathcal{L}_{sitcalc}$. Then*

$$\mathcal{D} \models executable(do([\alpha_1, \dots, \alpha_n], S_0))$$

iff

$$\mathcal{D}_{S_0} \cup \mathcal{D}_{una} \models \bigwedge_{i=1}^n \mathcal{R}[Poss(\alpha_i, do([\alpha_1, \dots, \alpha_{i-1}], S_0))].$$

- This provides a systematic, regression-based method for determining whether a ground situation $do([\alpha_1, \dots, \alpha_n], S_0)$ is executable. Moreover, it reduces this test to a theorem-proving task *in the initial database* $\mathcal{D}_{S_0}$, together with unique names axioms for actions.

Example: Executability Testing

- Continue with our previous database example.
- Compute the legality test for the transaction sequence

$register(Bill, C100), drop(Bill, C100), drop(Bill, C100)$

which intuitively should fail because the first *drop* leaves *Bill* unenrolled in *C100*, so that the precondition for the second *drop* will be false.

- First compute

$$\begin{aligned} &\mathcal{R}[Poss(register(Bill, C100), S_0)] \wedge \\ &\mathcal{R}[Poss(drop(Bill, C100), do(register(Bill, C100), S_0))] \wedge \\ &\mathcal{R}[Poss(drop(Bill, C100), do(drop(Bill, C100), \\ &\quad do(register(Bill, C100), S_0)))], \end{aligned}$$

which is

$$\begin{aligned} &\mathcal{R}[(\forall p).prerequ(p, C100) \supset (\exists g).grade(Bill, p, g, S_0) \wedge g \geq 50] \wedge \\ &\mathcal{R}[enrolled(Bill, C100, do(register(Bill, C100), S_0))] \wedge \\ &\mathcal{R}[enrolled(Bill, C100, do(drop(Bill, C100), \\ &\quad do(register(Bill, C100), S_0)))]. \end{aligned}$$

- This yields

$$\begin{aligned} &\{(\forall p).prerequ(p, C100) \supset (\exists g).grade(Bill, p, g, S_0) \wedge g \geq 50\} \wedge \\ &true \wedge \\ &false \end{aligned}$$

- So the transaction sequence is indeed illegal.

Legality Testing: Another Example

- Consider next the sequence

$change(Bill, C100, 60), register(Sue, C200), drop(Bill, C100).$

- First compute

$$\begin{aligned} & \mathcal{R}[(\exists g')grade(Bill, C100, g', S_0) \wedge g' \neq 60] \wedge \\ & \mathcal{R}[(\forall p)prerequ(p, C200) \supset \\ & \quad (\exists g)grade(Sue, p, g, do(change(Bill, C100, 60), S_0)) \wedge g \geq 50] \wedge \\ & \mathcal{R}[enrolled(Bill, C100, do(register(Sue, C200), \\ & \quad do(change(Bill, C100, 60), S_0)))]. \end{aligned}$$

- This simplifies to

$$\begin{aligned} & \{(\exists g').grade(Bill, C100, g', S_0) \wedge g' \neq 60\} \wedge \\ & \{(\forall p).prerequ(p, C200) \supset \\ & \quad Bill = Sue \wedge p = C100 \vee (\exists g).grade(Sue, p, g, S_0) \wedge g \geq 50\} \wedge \\ & \{Sue = Bill \wedge C200 = C100 \vee enrolled(Bill, C100, S_0)\}. \end{aligned}$$

- So the transaction sequence is legal iff this sentence is entailed by the initial database together with unique names axioms for actions.

The Projection Problem

- Given an action sequence $\alpha_1, \dots, \alpha_n$ of *ground* action terms, and a query $Q(s)$ whose only free variable is the situation variable s , what is the answer to Q in that situation resulting from performing this action sequence, beginning with the initial world situation S_0 ?

- Define this formally as the problem of determining whether

$$\mathcal{D} \models Q(do([\alpha_1, \dots, \alpha_n], S_0)).$$

- This is the *projection problem*.
- $Q(do([\alpha_1, \dots, \alpha_n], S_0))$ will normally be regressive.
- So, by the Regression Theorem, regress $Q(do([\alpha_1, \dots, \alpha_n], S_0))$, and verify it in the initial situation with unique names axioms for actions.

Proj. Prob. Ex: Database Query Evaluation

- Consider again the transaction sequence

$$\mathbf{T} = \text{change}(\text{Bill}, C100, 60), \text{register}(\text{Sue}, C200), \text{drop}(\text{Bill}, C100).$$

- Suppose the query is

$$\begin{aligned} & (\exists st). \text{enrolled}(st, C200, \text{do}(\mathbf{T}, S_0)) \wedge \\ & \neg \text{enrolled}(st, C100, \text{do}(\mathbf{T}, S_0)) \wedge \\ & (\exists g). \text{grade}(st, C200, g, \text{do}(\mathbf{T}, S_0)) \wedge g \geq 50. \end{aligned}$$

- Regress this query.
- After some simplification, assuming that $\mathcal{D}_{S_0} \models C100 \neq C200$, we obtain

$$\begin{aligned} & (\exists st). [st = \text{Sue} \vee \text{enrolled}(st, C200, S_0)] \wedge \\ & [st = \text{Bill} \vee \neg \text{enrolled}(st, C100, S_0)] \wedge \\ & [(\exists g). \text{grade}(st, C200, g, S_0) \wedge g \geq 50]. \end{aligned}$$

- The answer to the query is obtained by evaluating this last formula in $\mathcal{D}_{S_0}$, together with unique names axioms for actions.

Planning in Artificial Intelligence

Giuseppe De Giacomo

Università degli Studi di Roma “La Sapienza”

Planning in AI

INTRODUCTION

AI Aims at Conceiving Systems that Act Autonomously

- Autonomy is one of the grand objectives of AI.
- Aims at building autonomous agents/robots that operate in changing, incompletely known, unpredictable environments.
- In other words: Aims at empowering the agent with the ability of
 - deliberating ...
 - how to act in the world ...
 - autonomously
- First person view: it is the agent/system that decides what to do



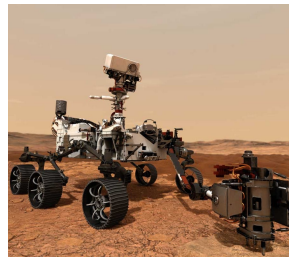
Examples of Autonomous Systems

Space Exploration

Delay in communication requires high-level of autonomy during the mission.

Planning and scheduling for temporal extended goals is a top research topic at NASA.

<https://mars.nasa.gov/mars2020/>



Interacting with Humans

Robots interacting with humans needs to properly schedule execution of complex actions that extends over time, as well as react to unexpected circumstances.



Autonomy is a Challenge in Business Process Management (BPM)

- BPM'21 Tutorial: Applications of Automated Planning for Business Process Management by Andrea Marrella and Tathagata Chakraborti



- BPM'21 Tutorial: RuM: Declarative Process Mining, Distilled by Anti Alman, Claudio Di Ciccio, Fabrizio Maria Maggi, Marco Montali and Han van der Aa



- BPM'21 Invited talk: Artificial Intelligence-based Declarative Process Synthesis for BPM by Giuseppe De Giacomo

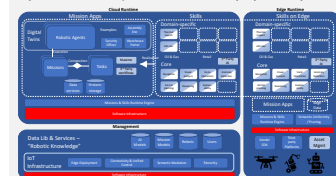


Autonomous Mobile Robots in Logistics

Complex multi-robot systems need highly synchronized behaviours to fulfil their job.

These robots need autonomously resolve unexpected clashes.

Sophisticated AMR platforms under study by industry



FinTech

- Smart Contracts Composition: systematic approach to compose many Smart Contracts (e.g. on Ethereum) from an extracted abstract specification to implement novel services for the user.

- Encoding Trading Strategies with Formal Methods:
 - Safety conditions: stop-loss, take-profit
 - Liveness conditions: order placement control
 - Interoperability with learning modules can forecast optimal order amounts
- Security & Fraud Detection: Process Mining to discover suspicious patterns of illegal activities e.g. money laundering
- Many AI + FinTech Academic Venues!
 - ICAPS's FinPlan workshop
 - ICAI's FinTech track



CHASE J.P.Morgan 2021 AI Faculty Award

Smart Manufacturing and Digital Twins

- Manufacturing as a service products to be manufactured are not known in advance and each product may differ from the products manufactured immediately before and immediately after it
- Analogies with Service Composition and Orchestration: synthesize the orchestrator
- Digital Twins platforms offer infrastructure to deploy orchestrations
- Automated exception handling is crucial



G. De Giacomo, R. Felli, B. Logan, F. Patrizi, S. Sardina: Situation Calculus for Controller Synthesis in Manufacturing Systems with First-Order State Representation. AIJ2021
G. De Giacomo, M. Favaria, F. Leotta, M. Maccolla, L. Sile: Digital Twins Composition via Markov Decision Processes. ITBPM2021

KR: Have a Model of Environment and Use it to Act

Knowledge Representation:

“Equip agent with a model of the environment it acts in”

Reasoning about actions (classical view):

- Capture model of the environment with a **logical theory**
 - Preconditions for agent actions
 - Effects of agent actions + solution to “Frame Problem”
- **Multiple interpretations of the theory**
 - Multiple possible instantiations of the environment (one of them the **correct one**, but we do not know which)
- Reasoning (skeptical)
 - based on **logical implication**

